GrocerSeeker Prototype Project

Introduction

GrocerSeeker is a new establishment that focuses on creating an online platform to connect local grocery **vendors** and **consumers**, providing a platform where fresh essentials are just a click away.

They would like to develop a new **Desktop Application** using **C# Programming Language** to manage their **user profile, listed product, and the transaction process**. The application must have several features such as **login authentication**, **viewing user profile**, **product list and management**, and **transaction process and history**. Below are some of the requirements that have been arranged by the analysts and designers.

Contents

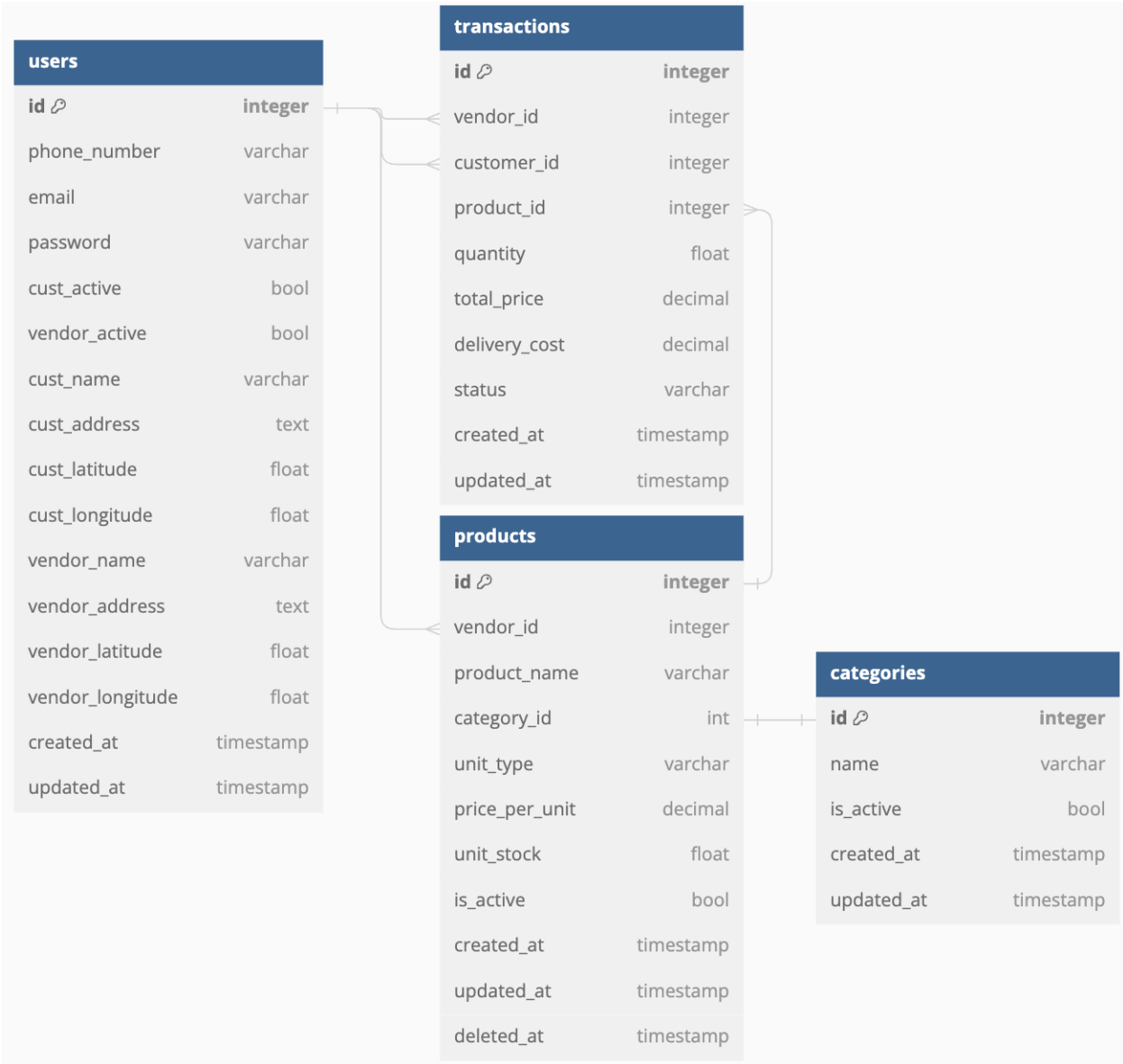
In this test project there are some files and usable resources:

1. GrocerSeeker_TP.pdf (Test project file)
2. GrocerSeeker_DB.sql (Database Script File)

Project Description and Tasks

- Project Criteria:
  - The project name should be "DESKTOP_I_[XX]", where XX is your PC number.
  - Time management is a crucial factor for the success of this project. So, manage your time wisely and submit all your works on time.
- Application User Interface:
  - The application wireframe is already provided. You are allowed to modify the application interfaces as long as it doesn't interfere with the business and application flow.
  - The scoring will be focused on the features of the application.
- Validation and Error Messages:
  - The application must run well with a proper validation and show useful error messages to the user.
  - Analyze the required validation(s) that is/are needed in the application. This is one of the scoring criteria for your analytical skills for the project. Any specific validation will be given in the project description.

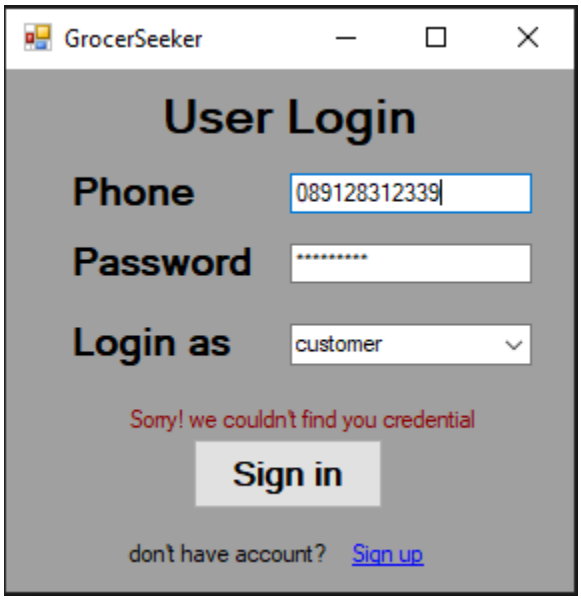
Entity Relationship Diagram



Entity-Relationship Diagram (ERD) is provided to help you understand the database structure of the requested GrocerSeeker Desktop Application. This diagram explains the concept and representation of the data model that is used in the database.

NB: The database implements **soft delete** technique; it means that records are not actually removed from the database. Instead, a **'deleted_at'** timestamp is set on the record.

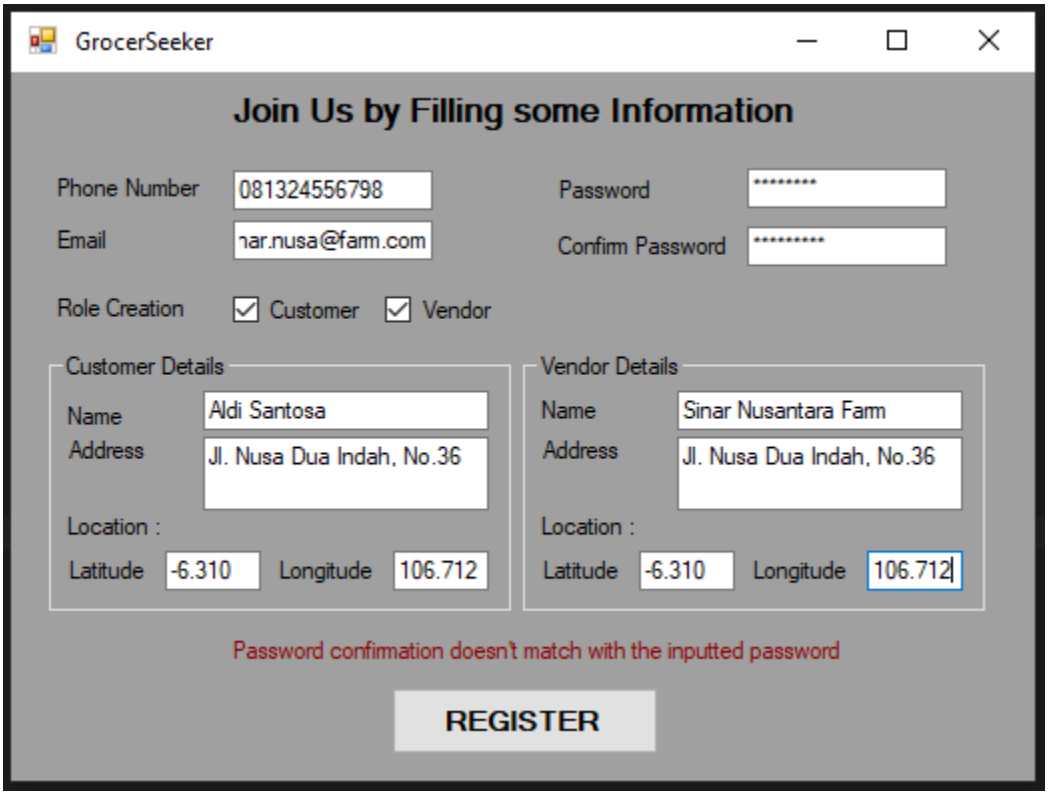
1. **Login Form**



This is the first screen that will be used by the user to login to the application. users can login to the application by filling in their **phone number , password,** and **role.**

- Validate the phone number is **digit** and **length** between **10-15 digits**.
- If the **combination exists** and the **user role is active** in the database, the user will be directed to the main screen of the application.
- If t**he combination exists but the user role is inactive in the database**, let the user know that is **unauthorized login**.
- If **the combination doesn't exist in the database**, give the error message.

2. Create New Account Form

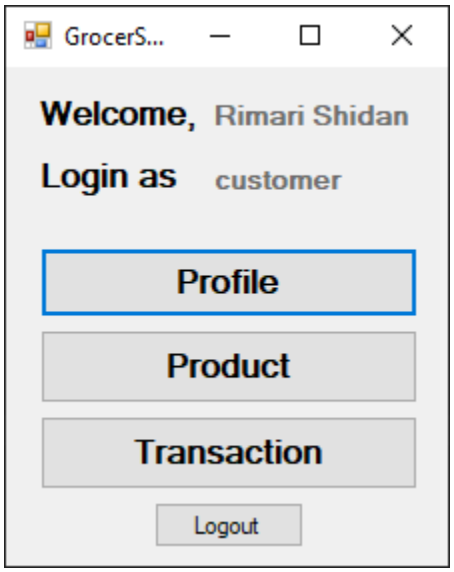


This Form is used to register new user by filling some information

- Phone number,
  - Required.
  - Validate the phone number is **digit** and **length** between **10-15 digits**.
  - Unique, only 1 that can exist in the database. Give an error message if duplicate.
- Password.
  - Required,
  - Must be a combination of uppercase, lowercase characters , and numbers with length minimum 8 characters. Give an error message if it doesn't meet criteria.
- Retype Password
  - Required,
  - Must be matched with the inputted password. Give an error message if it doesn’t match.
- Email.

    - Required,
    - Must be a proper email format , e.g. : xxx@xxx.xxx. Give an error message if it doesn't have a proper format.
- Role. Vendor and/or Customer.
    - Required, fill at least 1 role. (if unchecked, disable/hide the designated group).
    - If vendor is checked,
        - Show/enable to fill the vendor details, such as:
            - Vendor name (Required)
            - Vendor Address (Required)
            - Vendor Coordinate
                - Required
                - Validate it by having latitude and longitude, both values should be digits.
    - If customer is checked,
        - Show/enable to fill the customer details, such as:
            - Customer name (Required)
            - customer Address (Required)
            - customer Coordinate
                - Required
                - Validate it by having latitude and longitude, both values should be digits.

3. Main Form

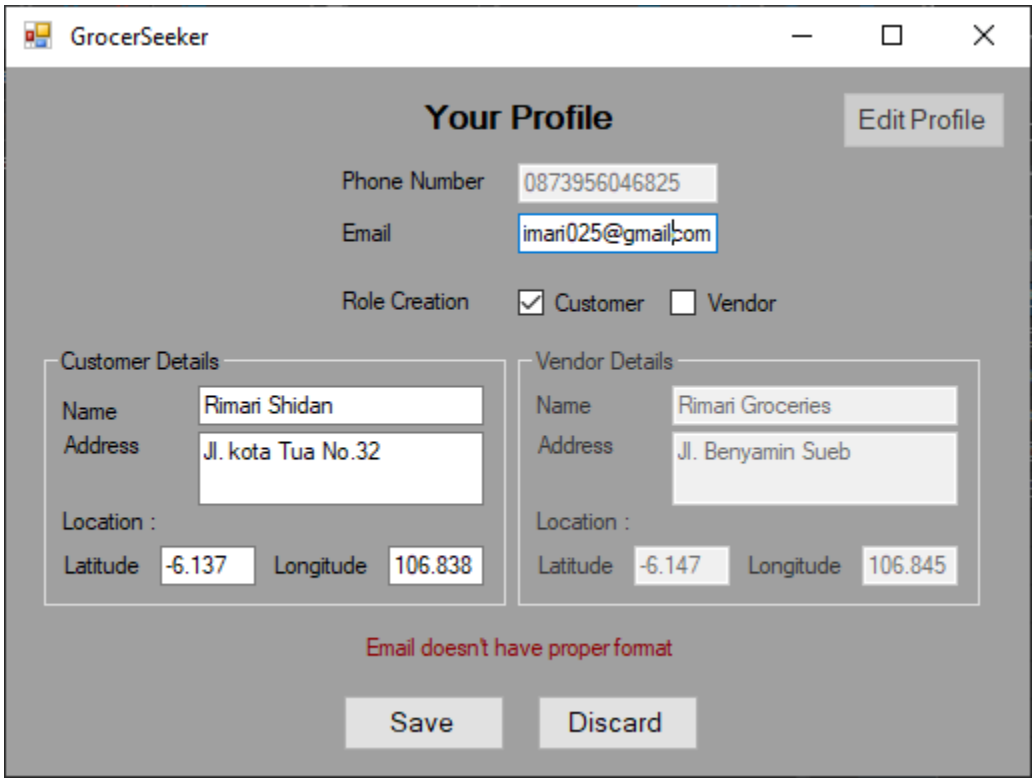


In this screen, user could navigate into other form

- Display the user's name and login role.

- Button Profile redirects user to Profile Form.
- Button Product redirects user to Product Form.
- Button Manage Transaction redirect user to Transaction Form.
- Button Logout redirect user back to login Form.

4. Profile Form



In this form it will show user information and could edit some information.

- Email.
  - Required,
  - Must be a proper email format , e.g. : xxx@xxx.xxx. Give an error message if it doesn't have a proper format.
- Role. Vendor and/or Customer.
  - Required. fill at least 1 role.
  - If vendor is checked,
    - Show/enable to fill the vendor details, such as:
      - Vendor name (Required)
      - Vendor Address (Required)
      - Vendor Coordinate
        - Required

        - Validate it by having latitude and longitude, both values should be digits.
    - Show/enable to fill the customer details, such as:
        - customer name (Required)
        - customer Address (Required)
        - customer Coordinate
            - Required
            - Validate it by having latitude and longitude, both values should be digits.

When users change their role by **unchecking their active role and saving it**, it will impact other processes and will be forced to login again.

- Inactive customer role, it will automatically cancel their pending transaction.
- Inactive vendor role, it will automatically cancel their pending transaction and their product listing will be inactive. If they reactivate their vendor role, they need to activate it manually.

5. Product Form

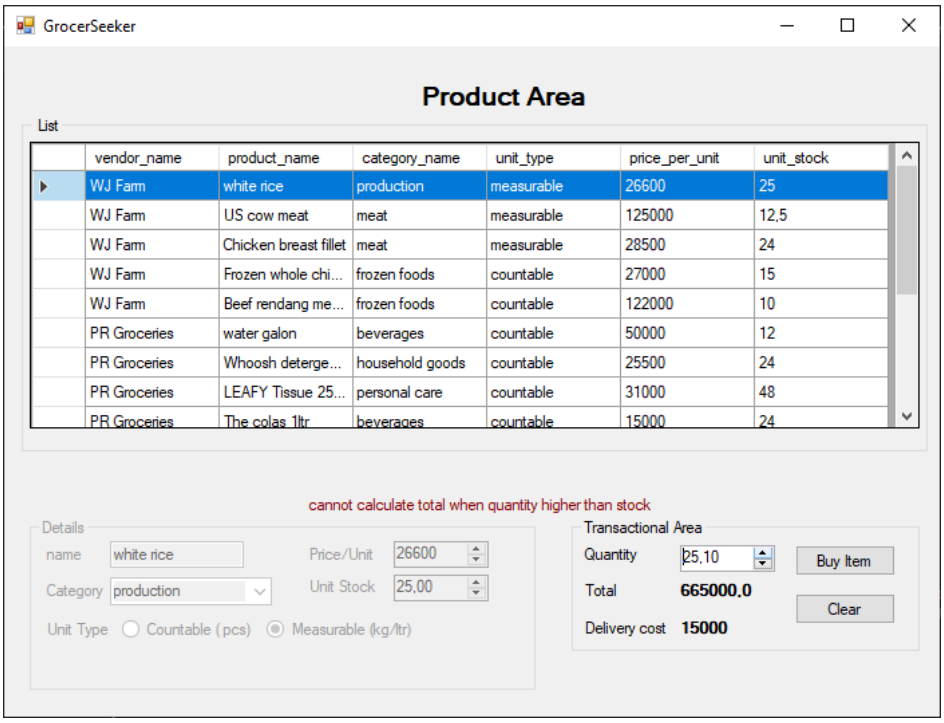


image illustration of product page for customer role

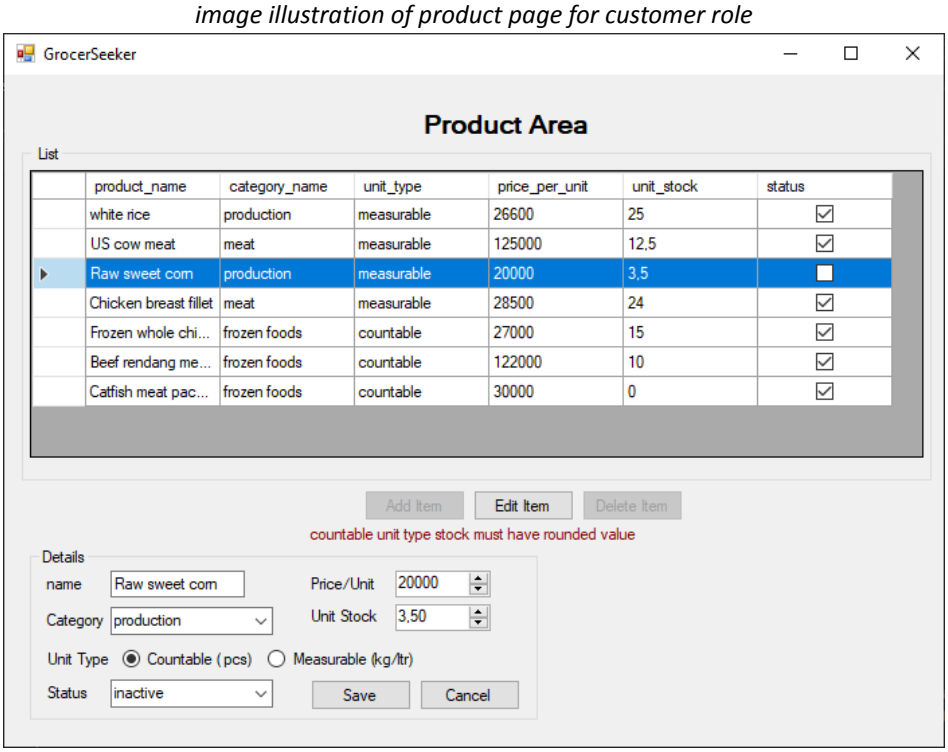


image illustration of product page for vendor role

In this form, it will show different data depending on the login role.

- Customer Role
    - The list will show all the products from active vendors and active products determined by active status and stock more than 0. If you have vendor roles active, then the list won't show your product listing.
    - Show the item details when you click an listed item and show the buy button.
    - When user is making the transaction,
        - ask the quantity and validate the input won't exceed the remaining availability and must be digits.
        - Calculate the total transaction based on formula $quantity \ * \ price\ per\ unit$
        - Delivery cost is calculated by charging the user **based on the distance, 15000 per 100 KM.**
        - **Distance** can be calculated using the **difference** between the **vendor's coordinate and the customer's coordinate** by applying the **haversine formula**.

$$a = sin^2(\Delta\varphi/2) + cos\,\varphi1 * cos\,\varphi2 * sin^2(\Delta\lambda/2)$$
$$c = 2 * atan2(\sqrt{a}, \sqrt{(1 - a)})$$
$$d = R * c$$

where:

- φ is latitude
- λ is longitude
- R is the Earth's radius (mean radius = 6371 km)
- d is the distance between the two points

        - When a customer clicks the Buy button, it will record the transaction and move it to the pending transaction.
        - Make sure when customers **finalize** the buying process, **quantity must be higher than 0** and **deduct the stock** availability.
        - Customers can **only have 10 pending transactions** when attempting to buy a new product. Either waiting for the pending approval by the vendor or the customer declining the pending transaction.
- Vendor Role
    - The list will show its own listed items that aren't deleted.

- Show the item details when you click an listed item and show the edit button and delete button.
- When you click the add or edit button. You need to fill the information required with some condition.
    - Product name, required.
    - Item category, required and select one category, the **category** must be **binded** from the **table categories with active status**.
    - Unit type, choose between countable (pieces, items) and measurable (kg, litre).
    - Price per unit , required and should be digit.
    - Stock, required and input digits based on the unit type where countable should be integer and measurable should be able to input float data type.
    - Status should choose between active and inactive, the default value is active.
- When a vendor successfully adds a product, make sure its stock is higher than 0.

6. Transaction Form

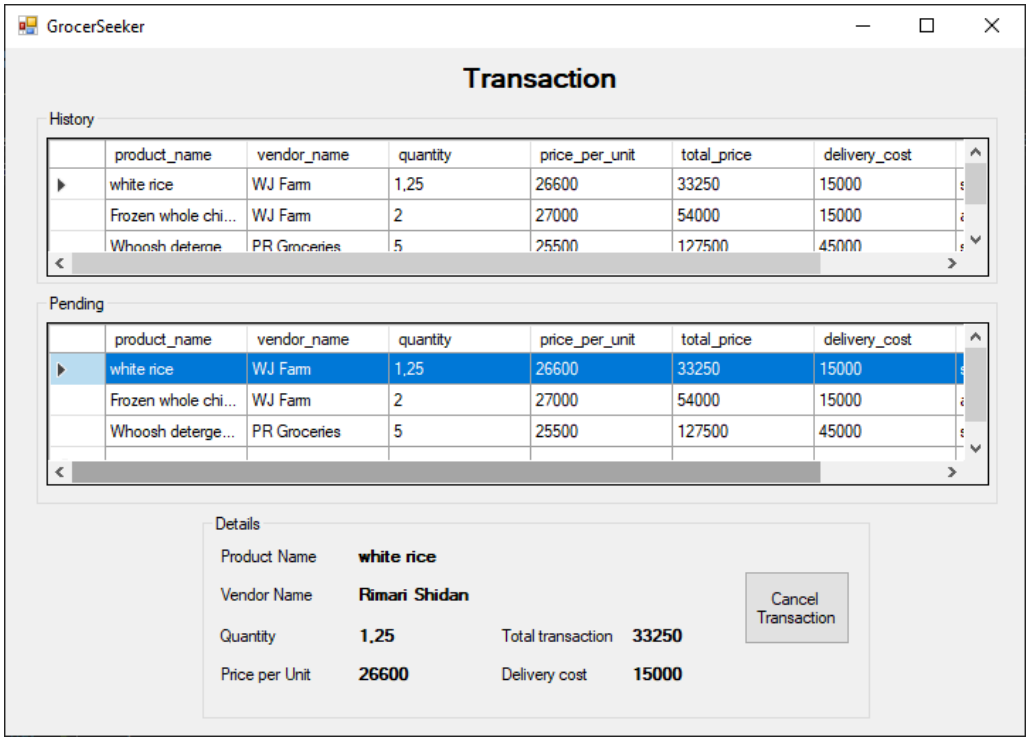


image illustration of transaction page for customer role

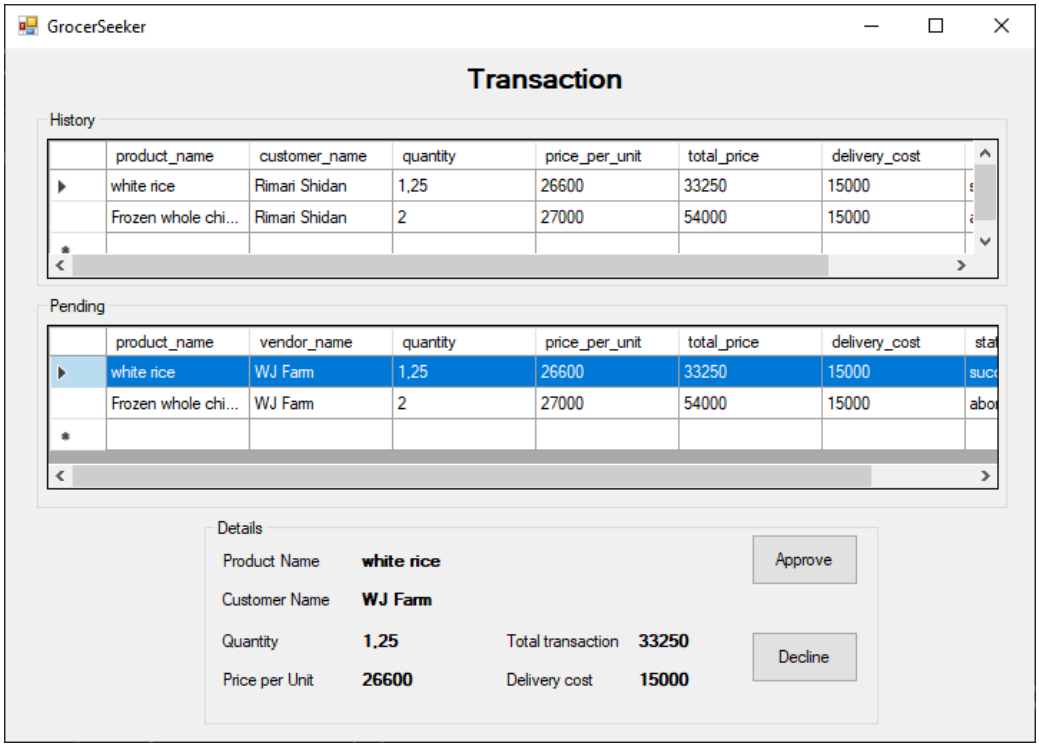


image illustration of transaction page for vendor role

In this form, it will show different data depends on login role

- Customer Role
    - There will be 2 lists shown in this page, transaction history and pending transaction list.
    - Transaction **history** will show the **successful** and **failed transaction** that was already made.
    - Pending transaction list will show the **pending** transaction that has already been made, when you click the pending transaction , it will show/enable **cancel transaction button**.
    - Show the details of the selected transaction such as product name, vendor name, bought quantity, price/unit, total, and delivery cost.
    - When the user confirms to cancel the transaction, the status will be updated to failed and moved to the transaction history. The product will be restocked based on the cancelled quantity.
- Vendor Role
    - There will be 2 lists shown in this page, transaction history and pending transaction list.
    - Transaction **history** will show the **successful** and **failed transaction** that was already made. When the user clicks a row, it will fill in the details.
    - Pending transaction list will show the **pending** transaction that has already been made, when you click the pending transaction , it will show/enable the **approve button and decline button**.
    - Show the details of the transaction such as product name, customer name, bought quantity, price/unit, total, and delivery cost.
    - When the user confirms to **approve** a transaction, the status will be updated to **success** and move it to the transaction history.
    - When the user confirms to **decline** the transaction, the status will be updated to **failed** and move it to the transaction history. The **product will be restocked** based on the cancelled quantity.s